Report on the Assignment 2: SQL and noSQL Solutions for e-commerce

1. Abstract

This report describes the stages of designing and implementing a scalable solution for storing and analyzing large volumes of data for a global e-commerce company. Various data models for a relational DBMS (PostgreSQL) and noSQL databases (MongoDB, Neo4J) are considered, along with a custom, improved model. The report covers data cleaning, data loading, the implementation of analytical queries, and testing aimed at comparing the performance of the solutions.

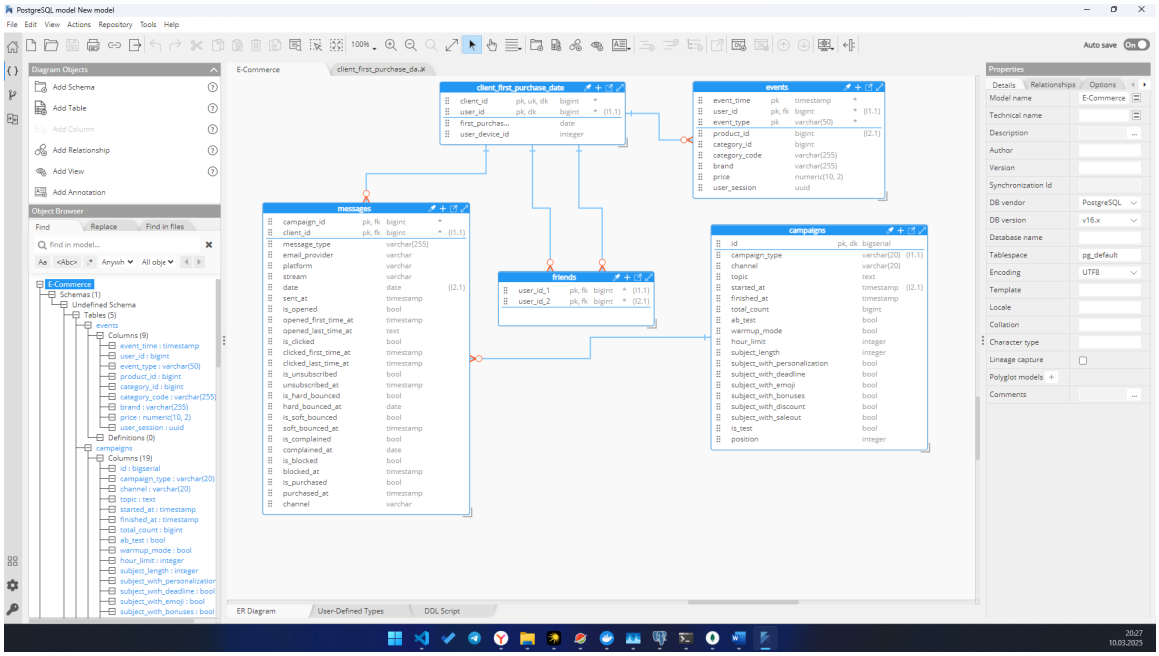
2. Data Modeling

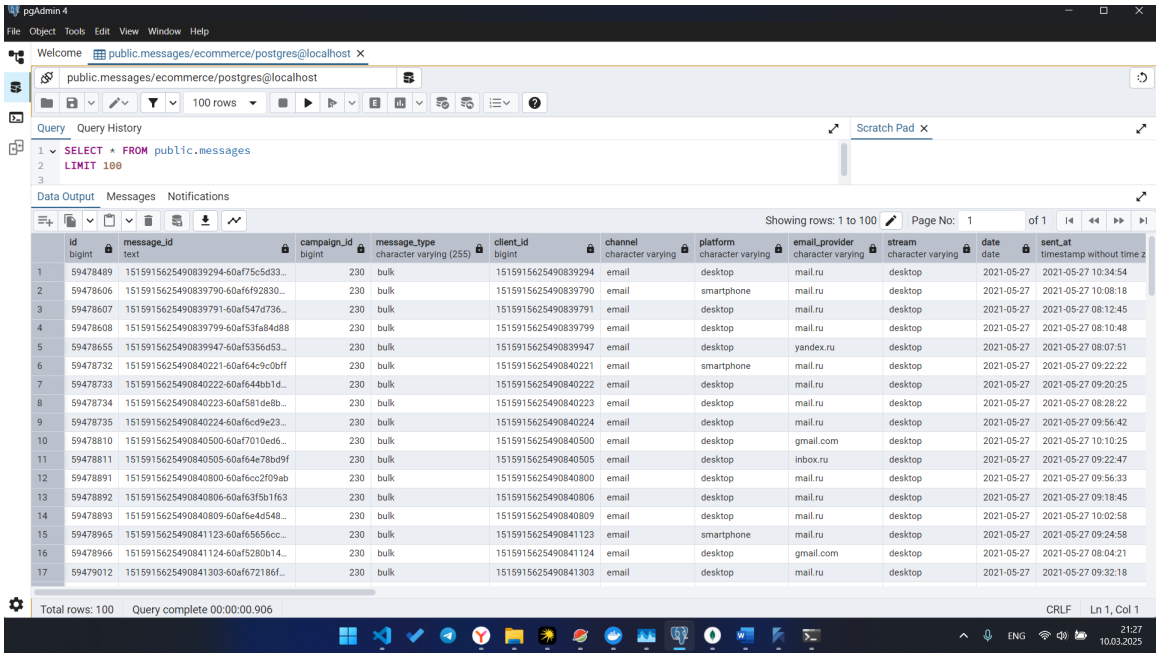
2.1 PostgreSQL (SQL)

Description:

An optimal relational schema has been developed to store information about transactions, products, users, and campaigns. The schema is designed for high-speed data retrieval through normalization and indexing of key fields.

Data Schema:





Data Loading Script:

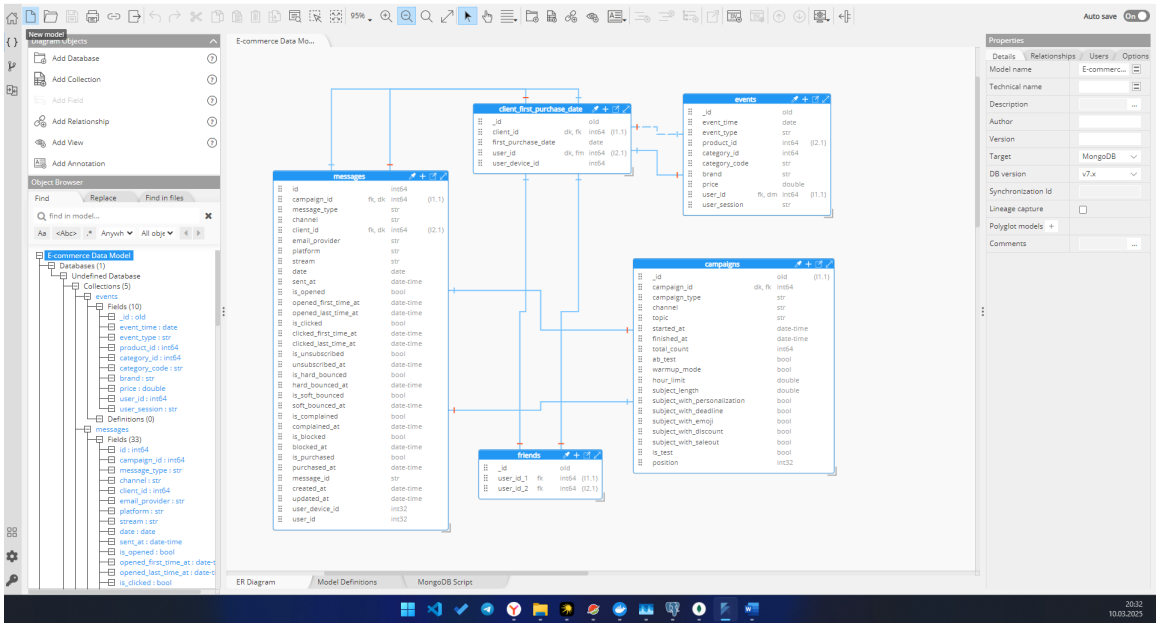
Loading the data are provided in the file `**/scripts/load_data_psql.sql**`.

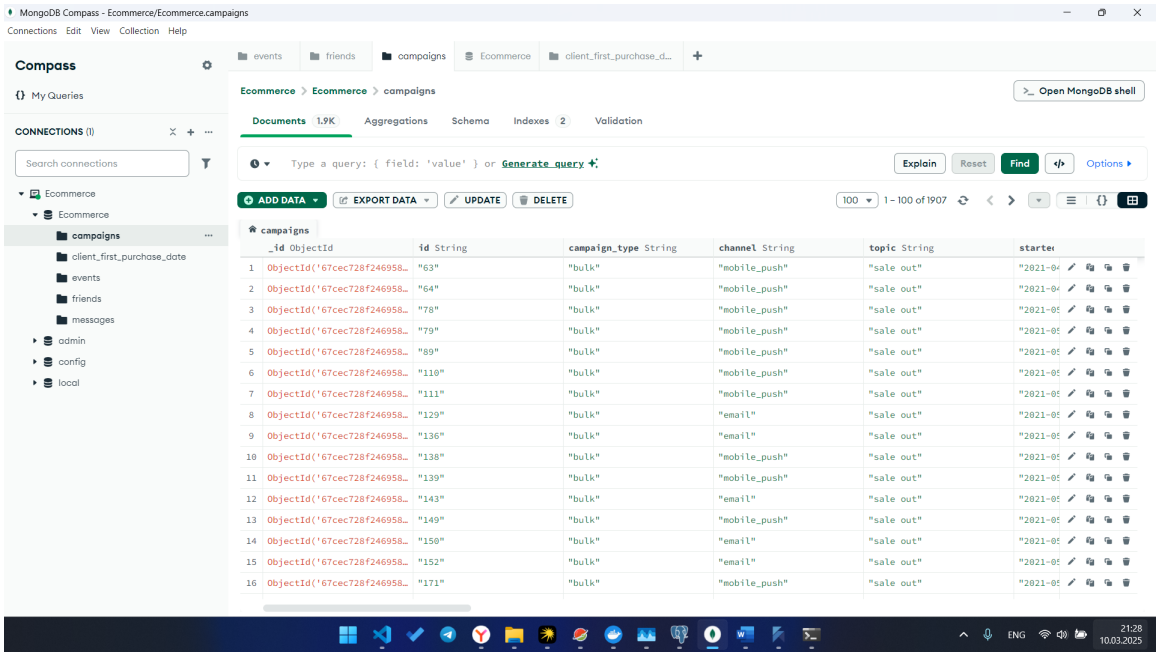
2.2 MongoDB (noSQL)

Description:

For MongoDB, a schema has been developed that uses collections to store documents reflecting the structure of transactions, user profiles, social interactions, and campaigns. The model is designed with an emphasis on data flexibility and the possibility of horizontal scaling.

Data Schema:





Data Loading Script:

Loading the data are provided in the file `**/scripts/load_data_mongodb.js**`.

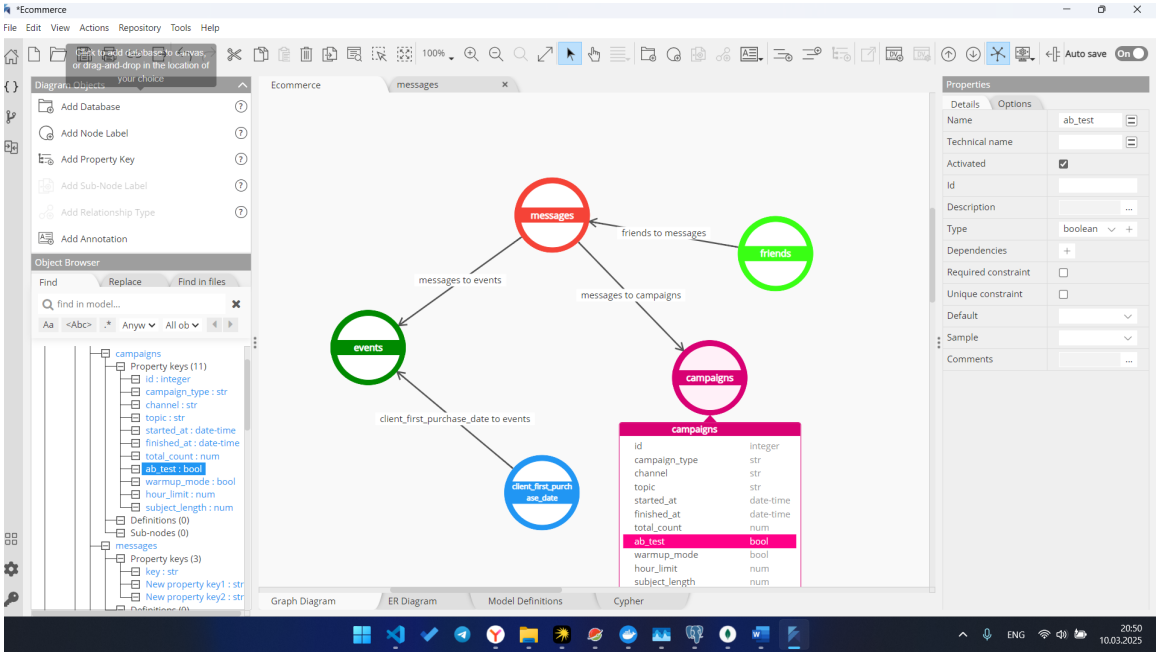
2.3 Neo4J (Graph Database)

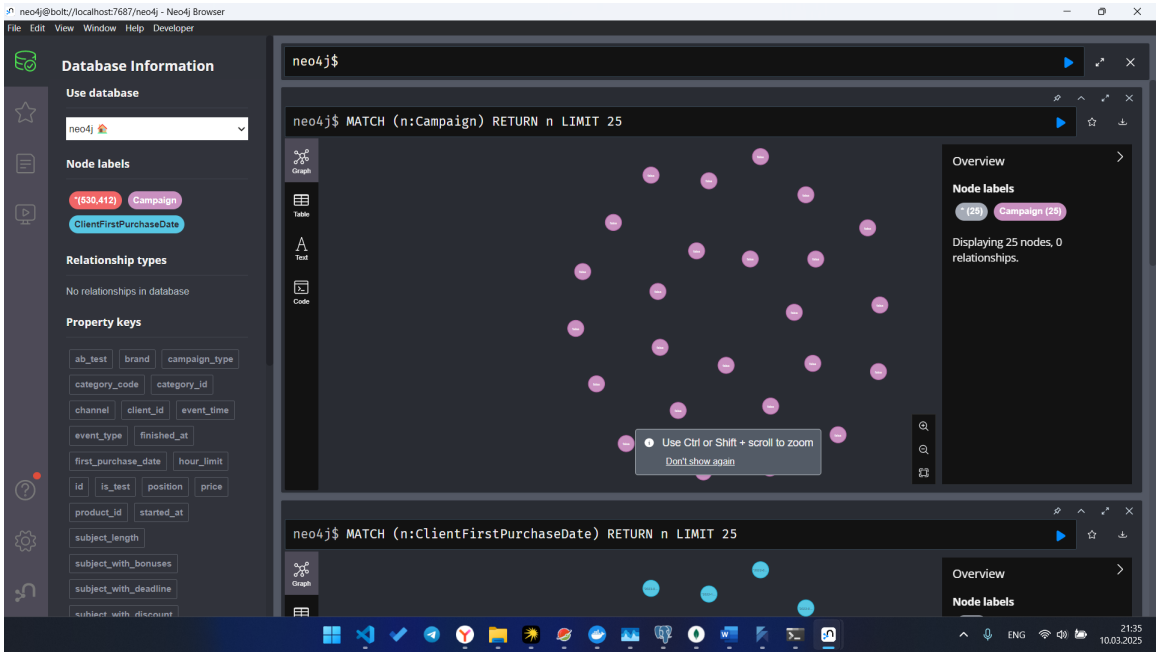
Description:

The data model for Neo4J is presented as a graph where nodes represent users, products, and campaigns, and edges represent relationships between them (e.g., a user's interaction

with a campaign or product). This model allows for effective analysis of social connections and transaction chains.

Data Schema:





Data Loading Script:

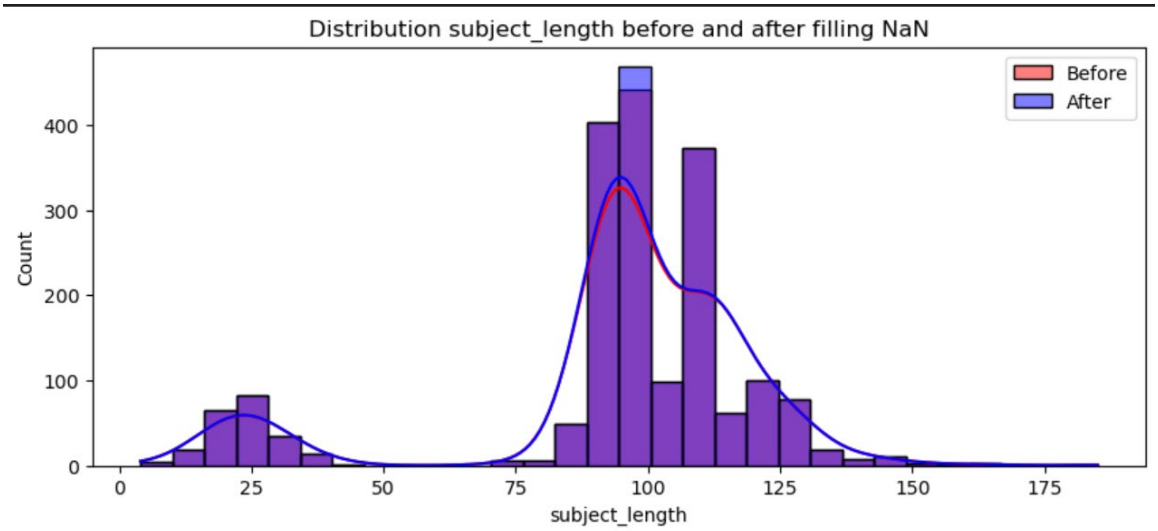
Loading the data are provided in the file `**/scripts/load_data_neo4j.cypher**`.

2.5 Data Cleaning

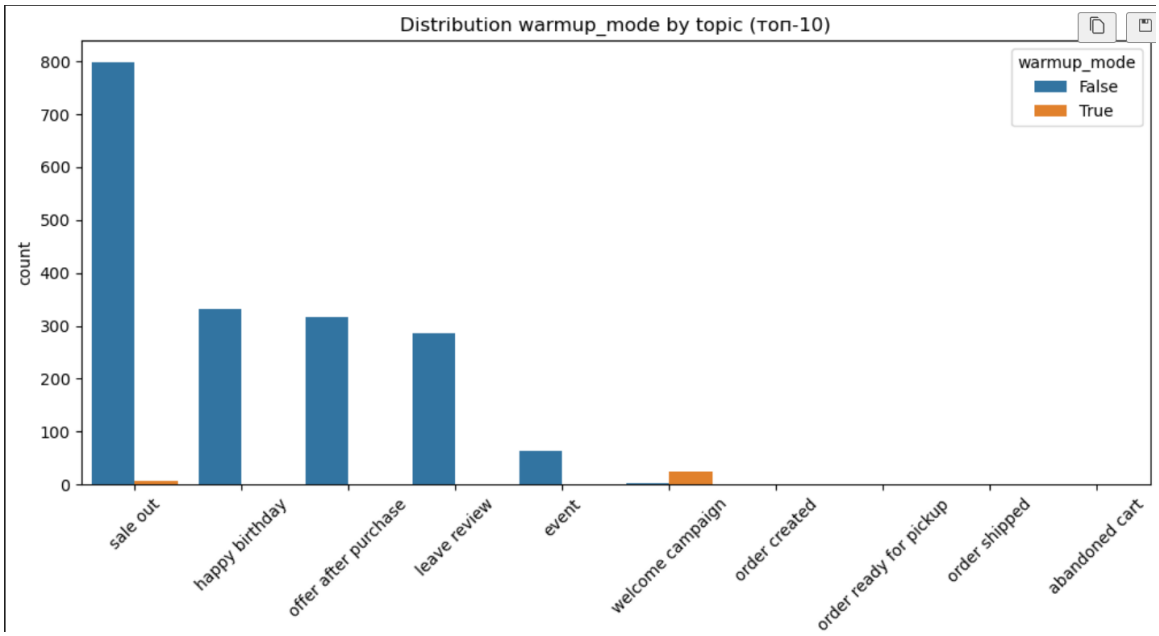
Description:

To ensure correct data loading, a cleaning script has been implemented. This script processes the raw dataset, removes duplicates, handles missing values, and brings the data into a consistent format.

Example filling columns (**subject_length**) after filling with **median.**



Example filling columns (**warmup_mode**)



Script:

The code is located in the file `**/scripts/clean_data.py**`.

3. Experimental Results

3.1 Analytical Tasks

For each of the three models (PSQL, MongoDB), the following analytical tasks were performed:

- **1. Campaign Impact Analysis:**

- Objective: Determine how effectively campaigns attract customers to make purchases.

- Queries:

- **PostgreSQL** – script `/scripts/q1.sql`

This SQL query retrieves the top 10 marketing campaigns with the highest purchase rate. It joins the **`campaigns`** table with a subquery that calculates the total purchases per campaign and client from the **`messages`** table. The query then aggregates data to count unique clients reached and total purchases made per campaign. It calculates the purchase rate as the percentage of purchases relative to reached clients, filters out campaigns with no clients, and orders the results in descending order of purchase rate. And the result will saved **`/output/Psql_q1_result.text`**

ID	Campaign Type	Channel	Total Count	Is Test	Position	Clients Reached	Purchases Made	Purchase Rate (%)
29	transactional	email	2006.5	False	0	20888	432	2.07
32	transactional	email	2006.5	False	0	24532	429	1.75
27	transactional	email	2006.5	False	0	26208	415	1.58
28	transactional	email	2006.5	False	0	948	15	1.58

- **MongoDB** – script `/scripts/q1.js`

This MongoDB aggregation pipeline filters campaigns of type "**bulk**," projects relevant fields, calculates the purchase rate (purchases made divided by total count, handling division by zero), sorts campaigns by **purchase rate** in descending order, and limits the output to the top 100 results. There is no **purchase_rate** because I am having problems with aggregation with the **messages** collection. And the result will saved **`/output/Mongo_q1_result.text`.**

_id	campaign_type	channel	total_count	purchase_rate
67cec728f246958dc06b1a11	bulk	email	96939	
67cec728f246958dc06b1a21	bulk	email	131583	
67cec728f246958dc06b19f1	bulk	email	6559	

- **2. Personalized Product Recommendation:**

- Objective: Identify the top products for personalized recommendations on the homepage.

- Queries:

- **PostgreSQL – script `/scripts/q2.sql`**

This SQL query identifies the top 10 users who purchased the same product more than once. It counts interactions (views, cart additions, and purchases) per user and product, filters for products with at least two purchases, and sorts the results by user, purchase count (descending), and total interactions (descending).

User ID	Product ID	Interaction Count	Purchase Count
546807372	1005204	9	2
546808271	4100151	11	2

- **MongoDB – script `/scripts/q2.js`**

This MongoDB aggregation pipeline finds the top 10 most-viewed product categories and brands per user. It filters for "view" events, groups data by **`user_id`**, **`category_code`**, and **`brand`**, counts the views, sorts by view count in descending order, and returns the top 10 results.

{'user_id': '557442625', 'category_code': 'apparel.shoes', 'brand': 'respect'}	1353
{'user_id': '549029080', 'category_code': 'electronics.smartphone', 'brand': 'apple'}	657
{'user_id': '563605520', 'category_code': 'furniture.living_room.sofa', 'brand': 'rals'}	540
{'user_id': '561174161', 'category_code': 'electronics.smartphone', 'brand': 'samsung'}	536

- **3. Full-Text Search on Product Categories:**

- Objective: Determine which products are retrieved when performing a full-text search using keywords from the product category.

- Queries:

- **PostgreSQL** – script `**/scripts/q3.sql**`

This SQL query retrieves a distinct list of 10 products with their category, brand, price, and a **text search vector** (`tsvector`) generated from the category code for full-text search in English.

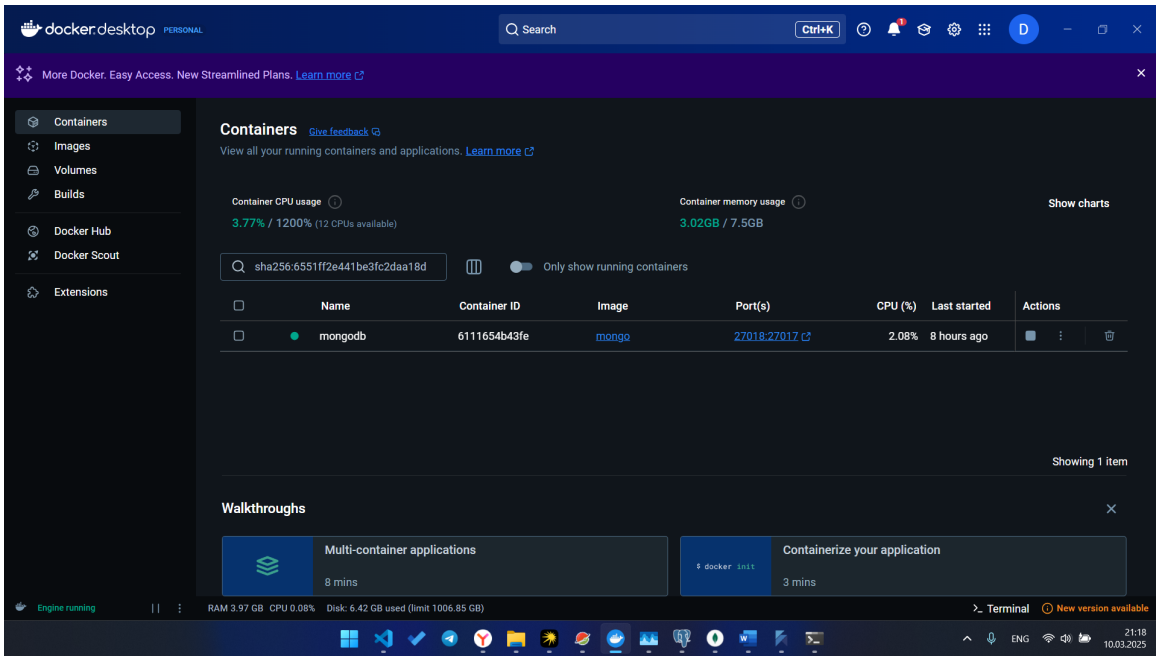
Product ID	Category Code	Brand	Price	Tsvector
1000894	electronics.smartphone	texet	73.31	'electronics.smartphone':1
1000978	appliances.kitchen.refrigerators	samsung	231.64	'appliances.kitchen.refrigerators':1
1000978	appliances.kitchen.refrigerators	samsung	300.91	'appliances.kitchen.refrigerators':1

- MongoDB – script `/scripts/q3.js`

There were issues with converting data to text and full-text index, when implementing this query in MongoDB.

3.2 Testing and Benchmarking

- **Test System Parameters:**
  - OS: (Windows 11 Pro)
  - RAM: (16GB)
  - CPU: (Intel Core i7 2.6 GHz)
- **Software**:
  - **PostgreSQL**: (PostgreSQL 16.8)
  - **MongoDB**: (MongoDB 8.0.5)
  - **Mongosh**: 2.3.8
  - **Neo4J**: (Neo4J 5.24.0)
  - **Neo4j Desktop**
  - **Docker**: 27.5.1 - I used Docker to run a MongoDB server and connected to it through MongoDB Compass.



Testing Methodology:

Each analytical task was executed 5 times per database. The execution time was measured using the `**time**`, and the data was collected to compute the average execution time and the standard deviation.

Screenshots and Results:

```
|            33 | transactional   | email        |        2006.5 | False
+---------------+-----------------+--------------+---------------+----------

Average execution time: 12.8170 seconds
Standard deviation: 0.4117 seconds
```

All screenshots and logs of test results are included in the report. The results are presented in a table showing the average execution times and standard deviations for each query.

Chart:

I was unable to understand the instructions for downloading the reference article, so I created a different graph instead.



Fig 7. Located - `**/output/query_executing_times.png**`

4. Discussion

4.1 Comparison of Data Models

- **PostgreSQL (SQL):**
  - Advantages: Reliability of transactions, well-structured data, and support for complex queries.
  - Disadvantages: Limited flexibility when modifying the schema and scalability challenges when dealing with petabytes of data.
- **MongoDB (noSQL):**
  - Advantages: Flexible data structure, horizontal scalability, and fast processing of unstructured data.
  - Disadvantages: Limited transactional capabilities and challenges in implementing complex queries when data is highly interconnected.
- **Neo4J (Graph Database):**
  - Advantages: Excellent representation and analysis of social connections, and fast execution of queries focused on entity relationships.
  - Disadvantages: Less efficient when processing large volumes of unstructured data and difficulties in horizontal scaling.

4.2 Final Conclusions

Based on the analysis and testing, it can be concluded that the choice of an optimal data model depends on the specific requirements of the company:

- For transactional operations and complex analytical queries, relational databases (**PostgreSQL**) may be preferable.

- For flexibility and scalability, especially when dealing with unstructured data, **MongoDB** can be the optimal choice.

- For analyzing social connections and user interactions, graph databases such as **Neo4J** are more effective.

This assignment was particularly challenging for me because I had very little prior experience with databases. The task required me to delve into different types of databases, their structures, and how to design and implement solutions for each. However, this experience has been invaluable as I have learned a great deal about data modeling, database management systems (DBMS), and how to apply these skills to solve real-world problems. It was a rewarding and informative learning process that greatly expanded my understanding of database solutions.

Appendices

Data Cleaning Script: /scripts/clean_data.py

Data Loading Scripts:

- **PostgreSQL**: /scripts/load_data_psql.sql

- **MongoDB**: /scripts/load_data_mongodb.js

- **Neo4J**: /scripts/load_data_neo4j.cypher

Analytical Queries:

- **Task** 1: /scripts/q1.sql, /scripts/q1.js

- **Task** 2: /scripts/q2.sql, /scripts/q2.js

- **Task** 3: /scripts/q3.sql, /scripts/q3.js

Python scripts for analysis are located in the /scripts folder.